

COMP9123 26s1 Final Practice

USYD 各科往届题库



W1 Unit Introduction and Asymptotic Analysis

Q1. MCQ

Which function has the tightest growth rate $\Theta(n \log n)$?

- (A) $5n \log_2 n + 7n^{0.9}$
- (B) $n^{1.01}$
- (C) $n(\log n)^3$
- (D) $3n^2 + \log n$

Q2. Calculation

For
 for $i = 1$ to n step 3 do constant work; then set $j = n$ and while $j > 1$ set $j = \lfloor j/2 \rfloor$,

give a tight $\Theta(\cdot)$ bound for the total number of loop iterations and briefly justify it.

Q3. Short Answer

Give the formal definition of $f(n) = O(g(n))$. Then explain why constants and lower-order terms are ignored in asymptotic notation.

Q4. Calculation

Solve the recurrence $T(n) = 4T(n/2) + cn$ with $T(1) = d$ using the Master Theorem. State $a, b, f(n)$ and the final asymptotic bound.

Q5. Short Answer

A developer replaces a $\Theta(n^2)$ algorithm by a $\Theta(n \log n)$ algorithm. Is the new program guaranteed to be faster on every input size? Explain two practical factors that matter.

Q6. MCQ

Which proof technique is most naturally used to prove a loop-based scanning algorithm?

- (A) Cut property
- (B) Loop invariant
- (C) Pigeonhole principle
- (D) Universal hashing

Q7. Calculation

Find a tight bound for $\sum_{i=1}^n \sum_{j=1}^i 1$. Give one sentence explaining the triangular-sum argument.

Q8. Short Answer

Explain the difference between worst-case, expected, and amortised running time. Give one data-structure example for each where the distinction matters.

Q9. Long Question

Consider a procedure that starts at a leaf of a binary tree and repeatedly moves to the parent until it reaches the root. At each visited node u , it calls $\text{BAR}(u)$, which takes $O(\log n)$ time. If the tree has height h and n nodes, analyse the running time in terms of h and give the worst-case bound in terms of n .

Q10. Short Answer

State the standard three-part structure of a correctness proof for an algorithm-design exam question.

W2 Abstract Data Structures and Lists

Q1. MCQ

Which operation is $O(1)$ on an array list but generally $O(n)$ on a singly linked list?

- (A) Access element at index i
- (B) Insert after a known node
- (C) Remove first node
- (D) Test whether list is empty

Q2. Short Answer

Define an abstract data type. Why is it useful to separate the ADT specification from the implementation?

**Q3. Calculation**

An initially empty dynamic array doubles its capacity whenever full. If n appends are performed, what is the total number of elements copied? Briefly justify using total copied elements.

Q4. Short Answer

Compare array lists and linked lists for: random access, insertion at front, deletion of a known node, and cache locality.

Q5. Long Question

Given two singly linked lists A and B , design an iterative algorithm that weaves them into a new list by alternately taking one node from A then one node from B . If one input list runs out first, append the remaining nodes of the other list in order. Prove the relative order of nodes from each original list is preserved and analyse the running time.

Q6. Short Answer

Why can removing items from an array while scanning left-to-right accidentally skip elements? Describe a safe scanning rule.

Q7. Calculation

A list map is implemented as an unsorted singly linked list of n key-value pairs. Give worst-case costs for get, put when key may already exist, and remove.

Q8. MCQ

Which structure is most suitable if we only need to append items at the rear and remove them from the front?

- (A) Stack
- (B) Queue
- (C) Hash table
- (D) Binary search tree

Q9. Short Answer

What is a positional list? Give one reason sentinel nodes simplify the implementation.

Q10. Long Question

Given a singly linked list of n integers, design an $O(n)$ algorithm to split it into two lists: negative values and non-negative values, preserving the original relative order within each output list. Prove the order-preservation property.

W3 Stacks, Queues, and Cost Analysis**Q1. MCQ**

Which statement about stacks and queues is correct?

- (A) A stack is FIFO.
- (B) A queue is LIFO.
- (C) A stack is LIFO.
- (D) A priority queue removes the oldest item.

Q2. Short Answer

Describe how to check whether a string of parentheses is balanced using a stack. State the invariant and time complexity.

Q3. Long Question

Design a stack-based algorithm that removes adjacent duplicate characters from a string. For example, processing abbaca should return ca. Give pseudocode-level detail, state the stack invariant, and analyse the running time.

Q4. Calculation

A circular-array queue has capacity 8, front index 3 and size 5. Which array indices are occupied? If one dequeue and two enqueues follow, what are the new front and size?

Q5. Short Answer

Explain why a queue can be implemented with a singly linked list using head and tail pointers so that enqueue and dequeue are both $O(1)$.

Q6. Calculation

An array-list operation repeatedly removes the first element until the list is empty. If each removal shifts all remaining elements, give the tight total running time.

Q7. MCQ

For a dynamic array stack, which statement is most accurate?

- (A) Every push is worst-case $O(1)$.
- (B) Push is amortised $O(1)$ with occasional $O(n)$ resize.
- (C) Pop always costs $O(n)$.
- (D) Top changes the stack size.

**Q8. Short Answer**

Give one example of a problem where a deque is more natural than either a pure stack or pure queue.

Q9. Long Question

A program scans an XML-like stream containing opening and closing tags. Design an algorithm that returns true if the stream is closed in the correct nested order. Explain why the stack top is the only tag that matters when a closing tag is read.

Q10. Short Answer

A queue implemented by a circular array keeps variables *front* and *size*. Why is storing *size* useful for distinguishing full and empty states?

W4 Trees, Recursion, and Tree Implementation**Q1. MCQ**

In a general rooted tree, the depth of a node is:

- (A) the number of children it has
- (B) the number of edges from the root to it
- (C) the height of its deepest leaf
- (D) the size of its subtree

Q2. Short Answer

Define depth, height, leaf, ancestor, and subtree size for a rooted tree.

Q3. Calculation

A complete binary tree has height h where the root has height 0. Give the maximum number of nodes in terms of h , and solve for h as a function of n .

Q4. Long Question

Design a recursive algorithm that computes the size of every subtree in a binary tree and stores it at the corresponding node. Prove by induction that the stored values are correct and analyse the running time.

Q5. Short Answer

Describe preorder, postorder, and level-order traversal. Give one use case for each.

Q6. Calculation

For a binary tree with preorder A, B, D, E, C, F and inorder D, B, E, A, C, F , reconstruct the tree or describe its parent-child relationships.

Q7. MCQ

Which traversal is naturally implemented with a queue rather than recursion or a stack?

- (A) Preorder DFS
- (B) Inorder DFS
- (C) Postorder DFS
- (D) Level-order traversal

Q8. Short Answer

Explain why storing an arbitrary binary tree in the same compact array layout as a heap can waste space.

Q9. Long Question

Let $\text{Foo}(u)$ recursively visits both children of u and does $O(1)$ work at u . Prove by induction that $\text{Foo}(\text{root})$ runs in $O(n)$ time.

Q10. Calculation

Suppose a recursive tree algorithm does work $O(|T_u|)$ at each visited node u and recursively visits both children. Give the worst-case time on a skewed tree and on a perfectly balanced tree.

W5 Binary Search Trees and AVL Trees**Q1. MCQ**

An AVL tree requires the balance factor at every node to be:

- (A) exactly 0
- (B) in $\{-1, 0, 1\}$
- (C) at most 2 in absolute value
- (D) equal to subtree size

**Q2. Calculation**

Insert 10, 20, 30 into an empty AVL tree. Identify the imbalance case and the rotation used.

Q3. Calculation

Insert 30, 10, 20 into an empty AVL tree. Identify the imbalance case and describe the double rotation.

Q4. Short Answer

Why does a rotation preserve the BST property? Answer using the inorder order of keys.

Q5. Long Question

Design an augmented AVL tree that supports search, insert, delete in $O(\log n)$ and select the k th smallest key in $O(\log n)$. State the extra field, how rotations update it, and why select is correct.

Q6. Short Answer

Compare an ordinary BST and an AVL tree for search, insertion, and deletion in the best and worst cases.

Q7. Calculation

A node has left height 4 and right height 2 after an insertion in the left child's right subtree. What AVL case is this and what rotations are applied?

Q8. MCQ

Which statement about BST search is true?

- (A) It always takes $O(\log n)$ time.
- (B) It follows at most one child pointer at each node.
- (C) It must visit every leaf.
- (D) It does not depend on tree height.

Q9. Short Answer

Describe the three cases for deleting a key from a BST: leaf, one child, and two children.

Q10. Long Question

A map needs get, put, remove, predecessor, and range reporting. Choose between hash table, sorted array, and balanced BST. Justify the choice and give operation costs.

W6 Maps, Priority Queues, Heaps, and Sorting**Q1. MCQ**

For a binary min-heap stored in a 0-indexed array, the children of index i are:

- (A) $i/2$ and $i/2 + 1$
- (B) $2i$ and $2i + 1$
- (C) $2i + 1$ and $2i + 2$
- (D) $i + 1$ and $i + 2$

Q2. Calculation

Is the array (2, 5, 3, 9, 7, 4, 8) a valid min-heap? Check every parent-child relation.

Q3. Long Question

Design a data structure using two heaps that supports inserting a stream of numbers and reporting the current median after each insertion. State the heap invariant, how rebalancing works, and the cost per insertion.

Q4. Short Answer

Compare unsorted list, sorted list, and heap implementations of a priority queue for insert and remove-min.

Q5. Calculation

Starting with min-heap array (4, 9, 6, 15, 12, 8), insert key 3 and show the resulting heap array after up-heap operations.

Q6. Calculation

For the min-heap array (3, 9, 6, 15, 12, 8), perform delete-min and show the resulting heap array after down-heap operations.

Q7. Short Answer

Why does bottom-up heap construction run in $O(n)$ time even though a single down-heap can cost $O(\log n)$?

Q8. MCQ

Which sorting algorithm has worst-case $O(n \log n)$, is stable in its usual implementation, but needs $O(n)$ auxiliary space?

- (A) Merge sort
- (B) Selection sort
- (C) Quicksort
- (D) Heap sort



Q9. Short Answer

Explain why comparison sorting has a lower bound of $\Omega(n \log n)$, but counting/radix sort can be linear under extra

Q10. Long Question

You have k sorted linked lists, each of length m . Design an algorithm to merge them into one sorted list in $O(mk \log k)$ time. (Note: the priority queue contents.)

W7 Hashing

Q1. MCQ

With separate chaining and load factor $\alpha = n/N$, expected search time under simple uniform hashing is:

- (A) $O(\log n)$
- (B) $O(1 + \alpha)$
- (C) $O(n^2)$
- (D) $O(N/n)$

Q2. Short Answer

Explain the difference between worst-case and expected-case lookup in a hash table.

Q3. Calculation

A table has 17 buckets and uses $h(k) = k \bmod 17$. Insert keys 34, 18, 52, 21, 35 with separate chaining. List the non-empty buckets and their chains.

Q4. Long Question

Given a log of n user IDs, design an expected $O(n)$ -time algorithm to report whether any ID appears at least twice. State the hash-table invariant, how collisions are handled, and the worst-case caveat.

Q5. Short Answer

What is a load factor? Why do hash tables resize, and why is resizing considered amortised?

Q6. MCQ

Which collision-resolution method can suffer from primary clustering?

- (A) Separate chaining
- (B) Linear probing
- (C) Balanced tree buckets
- (D) Universal hashing alone

Q7. Calculation

An open-addressed table of size 11 uses linear probing with $h(k) = k \bmod 11$. Insert 22, 1, 13, 24, 35 in this order. Give the final table positions.

Q8. Short Answer

What is primary clustering in linear probing, and how can double hashing reduce it?

Q9. Long Question

Suppose a hash table doubles its number of buckets whenever the load factor exceeds 0.75. Prove that a sequence of n insertions has $O(n)$ total rehashing work, assuming each rehash reinserts all existing keys.

Q10. Short Answer

When should you prefer a balanced search tree over a hash table for implementing a map? Give two operations that require ordering.

W8 Graph Representations and Traversals

Q1. MCQ

For a sparse graph with n vertices and m edges, the usual adjacency-list space usage is:

- (A) $O(n)$
- (B) $O(m)$
- (C) $O(n + m)$
- (D) $O(n^2 m)$

Q2. Calculation

Run BFS from s on directed edges $(s, a), (s, b), (a, c), (b, c), (c, d), (c, e), (d, e)$. List the BFS layers.

**Q3. Short Answer**

Why does BFS compute shortest path distances in an unweighted graph? State the key invariant.

Q4. Calculation

Give one valid DFS preorder for the graph in Q2 if adjacency lists are processed alphabetically.

Q5. Short Answer

What is a topological ordering? State a necessary and sufficient condition for a directed graph to have one.

Q6. Long Question

Design an $O(n + m)$ algorithm to decide whether a directed graph has a cycle. You may use DFS colours or topological sorting. Prove correctness.

Q7. Short Answer

Define a strongly connected component. Briefly describe Kosaraju's algorithm.

Q8. MCQ

Which graph representation supports $O(1)$ edge-existence query but uses $O(n^2)$ space?

- (A) Adjacency list
- (B) Adjacency matrix
- (C) Edge list
- (D) DFS tree

Q9. Long Question

Given a directed graph and target t , design an algorithm to compute the shortest unweighted distance from every vertex to t . Explain the graph transformation.

Q10. Short Answer

Explain how to compute connected components in an undirected graph using BFS or DFS, and state the running time with adjacency lists.

W9 Shortest Paths and Minimum Spanning Trees**Q1. MCQ**

Dijkstra's algorithm requires:

- (A) all edge weights nonnegative
- (B) the graph to be a tree
- (C) all weights equal
- (D) the graph to be directed

Q2. Calculation

For undirected weighted edges $(1, 2, 4), (1, 3, 2), (2, 3, 1), (2, 4, 5), (3, 4, 8), (3, 5, 10), (4, 5, 2)$, compute one MST using Kruskal and its total weight.

Q3. Short Answer

State Dijkstra's relaxation rule and explain what a decrease-key operation does.

Q4. Long Question

Given a weighted directed graph with nonnegative weights and a target t , design an algorithm to compute shortest path distances from every vertex to t within $O((n + m) \log n)$.

Q5. Long Question

Given s, t and budget σ , call edge (u, v) feasible if it lies on some s - t path of length at most σ . Design an algorithm to find the feasible edge with maximum weight.

Q6. Short Answer

Give a counterexample showing that a shortest path between two vertices need not be contained in the graph's MST.

Q7. MCQ

Which property justifies Kruskal's algorithm?

- (A) Stack invariant
- (B) Cut property
- (C) Failure function
- (D) Load factor

**Q8. Calculation**

Run one step of Prim's algorithm from vertex 1 on the graph in Q2. Which edge is chosen first and why?

Q9. Short Answer

What is a disjoint-set union structure used for in Kruskal's algorithm? Give the purpose of find and union.

Q10. Long Question

A graph has a set S of special vertices. Design a shortest-path algorithm for the minimum length s - t walk that visits at least one special vertex. Use a multi-source or layered-graph idea and analyse complexity.

W10 Greedy Method**Q1. MCQ**

For interval scheduling where the goal is the maximum number of non-overlapping intervals, the classic greedy rule is:

- (A) earliest start
- (B) shortest interval
- (C) earliest finish
- (D) latest finish

Q2. Long Question

Prove the earliest-finish-time interval scheduling algorithm using an exchange argument.

Q3. Long Question

Tasks have processing times t_i and deadlines d_i . Design the heap-based greedy algorithm that maximises the number completed on time. Include the invariant and $O(n \log n)$ complexity.

Q4. Calculation

Apply the deadline-heap algorithm to tasks $(2, 2), (1, 3), (3, 5), (2, 6), (4, 7), (1, 8)$. Which tasks remain selected and how many are scheduled?

Q5. Short Answer

Why is sorting sets by increasing size not a correct exact algorithm for set cover? Describe what a counterexample must show.

Q6. Long Question

Construct a concrete counterexample for the set-cover heuristic: process sets in nondecreasing size and add a set if it contains an uncovered element. Show the heuristic output and an optimal output.

Q7. Short Answer

Explain the greedy-choice property and optimal substructure. Why are both needed?

Q8. MCQ

Huffman's algorithm repeatedly combines:

- (A) two largest frequencies
- (B) two smallest frequencies
- (C) first two symbols alphabetically
- (D) two deepest leaves

Q9. Short Answer

Give the cut property for MSTs and explain why it is a greedy-safe choice.

Q10. Long Question

A blockchain stores non-overlapping transactions, each with fixed start time and processing duration. Design an $O(n \log n)$ greedy algorithm to store as many transactions as possible and prove it.

W11 Divide and Conquer**Q1. MCQ**

The recurrence for merge sort is:

- (A) $T(n) = T(n/2) + O(1)$
- (B) $T(n) = 2T(n/2) + O(n)$
- (C) $T(n) = 4T(n/2) + O(n)$
- (D) $T(n) = T(n-1) + O(n)$

**Q2. Long Question**

Design a divide-and-conquer algorithm to count inversions in an array. Explain how the merge step counts cross inversions.

Q3. Calculation

Count the inversions in $[2, 4, 1, 3, 5]$ and show the cross-inversion reasoning.

Q4. Long Question

Design a divide-and-conquer algorithm to decide whether an array contains a majority element, meaning a value appearing more than $n/2$ times. Explain how candidates from the two halves are combined and verified.

Q5. Calculation

Using the majority-element divide-and-conquer idea, determine whether $[4, 2, 4, 4, 3, 4, 5, 4, 4]$ has a majority element.

Q6. Long Question

Given n distinct locations in an array and a distance d , design an $O(n \log n)$ algorithm to count pairs with absolute difference at most d . Also state the faster method if the array is already sorted.

Q7. Short Answer

Why does binary search on a linked list not achieve $O(\log n)$ time even if the list is sorted?

Q8. Calculation

Solve $T(n) = 2T(n/2) + O(1)$ and $T(n) = T(n/2) + O(n)$ using the Master Theorem.

Q9. Long Question

Design an $O(n \log n)$ divide-and-conquer algorithm to count important reverse pairs (i, j) such that $i < j$ and $A[i] > 2A[j]$. Explain how the merge step counts cross pairs.

Q10. Short Answer

When a divide-and-conquer algorithm outputs all pairs satisfying a property, why might the running time have an additional output-size term?

W12 Randomized Algorithms**Q1. MCQ**

Randomised quicksort is best classified as:

- (A) Monte Carlo with possible wrong output
- (B) Las Vegas with always correct output and random running time
- (C) deterministic dynamic programming
- (D) greedy approximation

Q2. Short Answer

For randomised algorithms, what is the expectation usually taken over: inputs or random choices? Explain.

Q3. Calculation

For quicksort on n distinct keys, what is the probability that two keys with ranks $i < j$ are compared? State the standard formula.

Q4. Short Answer

Explain why random pivot selection protects quicksort from a fixed adversarial input order.

Q5. Long Question

Design randomised quickselect for the k th smallest element. Prove that it is always correct and explain why its expected running time is $O(n)$.

Q6. Long Question

A universal hash family satisfies $\Pr[h(x) = h(y)] \leq 1/N$ for distinct keys x, y . If n keys are inserted into N buckets with chaining, use indicator variables to bound the expected number of colliding key pairs.

Q7. Short Answer

Define Monte Carlo and Las Vegas algorithms. Give one example of each or explain why randomised quicksort is one of them.

Q8. MCQ

Which statement is false?

- (A) Randomised quicksort is always correct.
- (B) Randomised quicksort has worst-case $O(n^2)$ possible.
- (C) Random pivot choice makes every partition exactly balanced.
- (D) Expected quicksort time is $O(n \log n)$.

Q9. Short Answer

Why does linearity of expectation not require independence? Explain how this helps in analysing randomised algorithms.

Q10. Long Question

For randomised quicksort, derive the recurrence for expected running time using the random pivot rank. You do not need to solve the recurrence fully, but explain why balanced pivots often enough lead to $O(n \log n)$ expected time.

USYD 各科往届题库



Answer Key

USYD 各科往届题库



W1

1. A.
2. $\Theta(n + \log n) = \Theta(n)$; first loop is linear and the later halving loop is logarithmic.
3. Use constants c, n_0 ; asymptotics track eventual growth.
4. $a = 4, b = 2$, so $n^{\log_b a} = n^2$ and $T(n) = \Theta(n^2)$.
5. No; constants, implementation, cache, and input range matter.
6. B.
7. $\Theta(n^2)$.
8. Worst-case BST search, expected hash lookup, amortised dynamic-array append.
9. $O(h \log n)$, worst $O(n \log n)$.
10. Algorithm, correctness, complexity.

W2

1. A.
2. ADT separates behaviour from representation.
3. Amortised $O(1)$; total copying over n appends is $O(n)$.
4. Array: fast access, slow front insert; linked list: local insert/delete fast, slow access.
5. Keep output tail; alternately append next node from A and B , then append remainder; each node moved once, $O(m + n)$.
6. Do not increment after deletion, or scan backwards.
7. get $O(n)$, put $O(n)$ if update needed, remove $O(n)$.
8. B.
9. A position is an abstract node handle; sentinels remove boundary cases.
10. Maintain negative and nonnegative tails; append each node to its bucket in scan order, so stability follows.

W3

1. C.
2. Push openings, pop matching closings, final stack empty; $O(n)$.
3. Stack stores reduced prefix; if next char equals top, pop, else push; final stack gives answer; $O(n)$.
4. Occupied 3, 4, 5, 6, 7; after one dequeue and two enqueues, front 4, size 6.
5. Enqueue at tail and dequeue at head.
6. $\Theta(n^2)$.
7. B.
8. Sliding-window min/max is a typical deque use.
9. Closing tag must match latest unmatched opening tag on top; $O(n)$.
10. It separates full from empty when front and rear indices coincide.

W4

1. B.
2. Standard rooted-tree definitions.
3. Max nodes $2^{h+1} - 1$, so $h = \Theta(\log n)$ for complete trees.
4. Postorder recursion computes child sizes first, then $1 + \text{left} + \text{right}$; induction proves correctness; $O(n)$.
5. Preorder root-first, postorder children-first, level-order by depth.
6. Root A ; left subtree root B with children D, E ; right chain C with right child F .
7. D.
8. Non-complete shapes create array gaps.
9. Each node contributes constant work once; induction over subtree size.
10. Skew $\Theta(n^2)$; balanced $\Theta(n \log n)$.

W5

1. B.
2. RR case; single left rotation, root 20.
3. LR case; left rotate 10 then right rotate 30.
4. Rotation preserves inorder sequence.
5. Store subtree sizes; update locally after rotations; select descends by left size, $O(\log n)$.
6. BST search/update $O(h)$ and can be $O(n)$; AVL guarantees $O(\log n)$ with rotations.
7. LR; left rotation at child then right rotation at node.
8. B.
9. Leaf, one-child splice, or replace by successor/predecessor then delete.
10. Balanced BST supports ordered queries; hash table lacks predecessor/range.

W6



1. C.
2. Valid: every parent is no larger than its children.
3. Max-heap for lower half and min-heap for upper half; sizes differ by at most one; median from roots; $O(\log n)$.
4. Unsorted insert $O(1)$ remove-min $O(n)$; sorted insert $O(n)$ remove-min $O(1)$; heap both $O(\log n)$.
5. (3, 9, 4, 15, 12, 8, 6).
6. (6, 9, 8, 15, 12).
7. Most nodes are near leaves; summed downheap heights are $O(n)$.
8. A.
9. Comparison lower bound; non-comparison sorts use key structure.
10. Heap stores current head from each list; total $O(mk \log k)$.

W7

1. B.
2. Worst collisions can be $O(n)$; expected assumes hashing/load assumptions.
3. Bucket 0: 34; bucket 1: 18, 52, 35; bucket 4: 21.
4. Insert seen IDs into a hash set; duplicate found on membership hit; expected $O(n)$, worst $O(n^2)$ with bad collisions.
5. $\alpha = n/N$; resizing keeps α bounded, amortised.
6. B.
7. Positions: $22 \rightarrow 0, 1 \rightarrow 1, 13 \rightarrow 2, 24 \rightarrow 3, 35 \rightarrow 4$.
8. Linear probing creates contiguous clusters; double hashing changes probe steps.
9. Doubling costs form a geometric series $O(n)$.
10. Prefer balanced trees for predecessor, successor, rank, and range.

W8

1. C.
2. $L_0 = \{s\}, L_1 = \{a, b\}, L_2 = \{c\}, L_3 = \{d, e\}$.
3. BFS processes nondecreasing distance layers.
4. One valid order: s, a, c, d, e, b .
5. Edge-forward order; exists iff DAG.
6. DFS colours: gray back edge iff cycle; $O(n + m)$.
7. Kosaraju: finish order, reverse graph, DFS again.
8. B.
9. Reverse edges and BFS from t .
10. Run BFS/DFS from each unvisited vertex; each component is one traversal tree; $O(n + m)$.

W9

1. A.
2. Edges (2, 3, 1), (1, 3, 2), (4, 5, 2), (2, 4, 5); total 10.
3. If $d[v] > d[u] + w$, update and lower priority.
4. Reverse graph and run Dijkstra from t ; $O((n + m) \log n)$.
5. Compute $dist_s$ and $dist_t$; feasible iff $dist_s[u] + w + dist_t[v] \leq \sigma$.
6. Triangle AB=2, BC=2, AC=3: shortest A-C is AC, but MST excludes AC.
7. B.
8. Edge (1, 3, 2).
9. DSU tests whether an edge joins different components.
10. Minimise $\min_{x \in S} dist_s[x] + dist_t[x]$ after two Dijkstra runs.

W10

1. C.
2. Exchange first optimal interval with earliest finisher.
3. Sort by deadline, keep max-heap of selected durations, remove longest if infeasible.
4. Selected T_1, T_2, T_4, T_6 ; count 4.
5. It must show heuristic output is larger than optimal.
6. One valid counterexample: singleton sets chosen before a full-cover set under the specified order; heuristic can use more sets than optimum.
7. Local choice must extend to some optimum and leave same-form subproblem.
8. B.
9. Lightest edge crossing any cut is safe.
10. Sort intervals by finish time and apply interval scheduling.

W11

1. B.
2. Merge count: right element before remaining left elements creates that many cross inversions.



3. Total 3.
4. Recursively find candidates; if halves disagree, count both candidates in the whole segment and keep one exceeding half.
5. 4 appears 6 times, so 4 is majority.
6. Sort then sliding window; sorted input gives $O(n)$.
7. Middle access costs linear.
8. Both recurrences are $\Theta(n)$.
9. During merge, advance right pointer while $A[i] > 2A[j]$ to count cross pairs, then merge sorted halves.
10. Output size K can force $\Omega(K)$ time.

W12

1. B.
2. Over random choices for a fixed input.
3. $2/(j-i+1)$.
4. The adversary cannot force a fixed bad pivot sequence.
5. Random pivot quickselect partitions and recurses only into the side containing rank k ; correctness by partition order, expected linear shrinkage.
6. Expected colliding pairs $\leq \binom{n}{2}/N$ by indicators.
7. Monte Carlo may err in bounded time; Las Vegas always correct with random time; quicksort is Las Vegas.
8. C.
9. $E[\sum X_i] = \sum E[X_i]$ without independence.
10. $E[T(n)] = n - 1 + \frac{1}{n} \sum_r E[T(r-1) + T(n-r)]$, leading to $O(n \log n)$.